Pi-CloudDOOM — Deployment Strategy Report

Project: InterviewPrep TN (Pi-CloudDOOM) **Branch:** `deployment/hybrid-openstack-azure-vercel` **Repo:** https://github.com/Mohamed-Fedi-Hamrouni/Pi-CloudDOOM **Production URLs:** - Frontend → https://interviewprep-tn.me (Vercel) - API → https://api.interviewprep-tn.me (OpenStack K8s, fronted by Cloudflare Tunnel + Traefik) - Auth → https://auth.interviewprep-tn.me (Keycloak in cluster) - AI services → `*.yellowocean-356174e3.francecentral.azurecontainerapps.io` (Azure Container Apps)

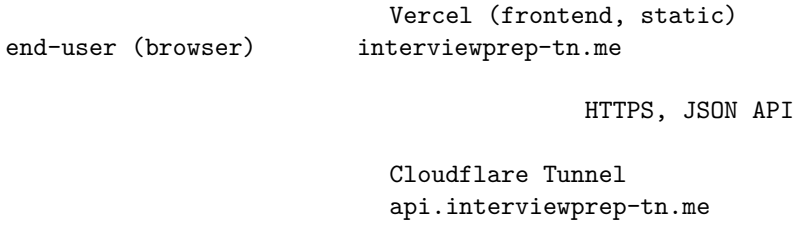
---

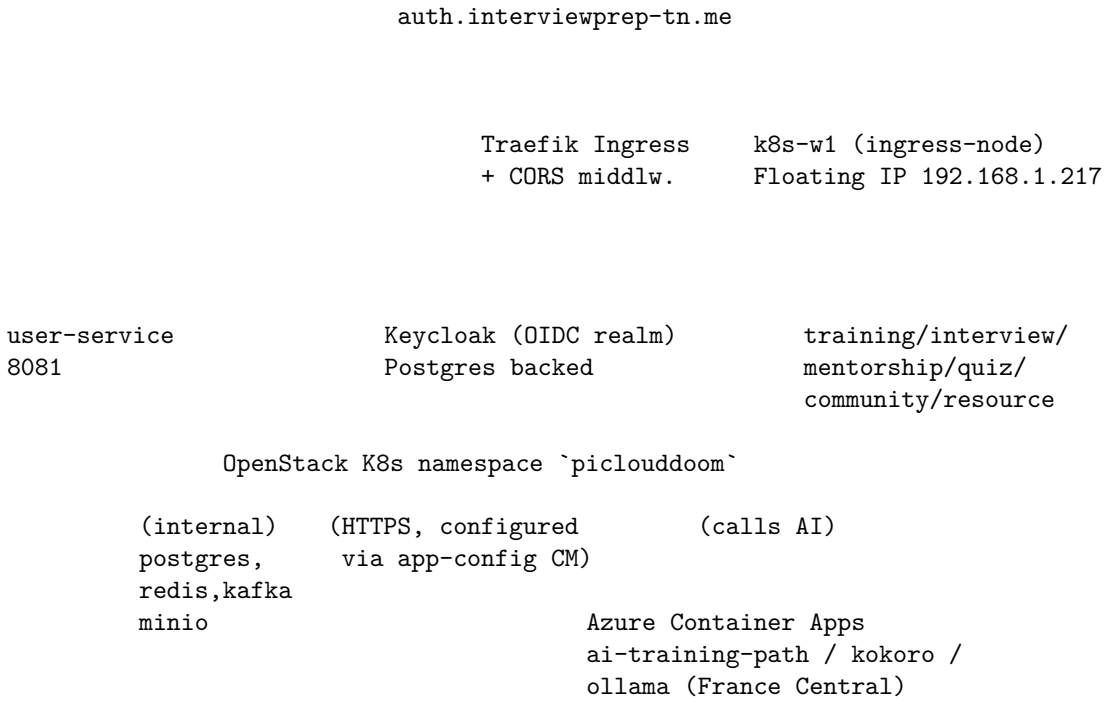
1. High-level deployment strategy

Pi-CloudDOOM uses a **hybrid, multi-cloud** deployment, splitting workloads by their natural fit:

Plane	Where	Why
7 Spring Boot microservices + stateful infra (Postgres, Redis, Kafka, MinIO, Keycloak)	**OpenStack** (private cluster, school infra)	Stateful, long-lived, free in-house compute
3 AI/ML services (ai-training-path, kokoro TTS, ollama LLM)	**Azure Container Apps** (France Central)	Scale-to-zero, pay-per-use GPU/CPU on demand, no infra babysitting
Angular 21 frontend (static)	**Vercel**	CDN edge, free preview URLs per PR, instant global delivery
Container registry	**DockerHub** (`docker.io/azizbna/pi-cloud*`, public)	Free, public, no `imagePullSecret` needed

This split lets us pin heavy stateful workloads in a controlled cluster while letting cold-starty AI services use a true serverless platform — and lets the frontend benefit from Vercel's CDN without coupling it to backend lifecycles.





---

2. Plane 1 — OpenStack Kubernetes (backend + stateful infra)

2.1 Cluster IaC — `infra/openstack/heat-cluster.yaml`

The whole cluster is described as an OpenStack **Heat** stack (declarative IaC):

- **5 nodes**: 1 control-plane (`k8s-cp1`) + 4 workers (`k8s-w1` to `k8s-w4`).
- **Placement across 3 compute hosts** (`compute2nacef`, `compute3ayoub`, `compute1yass`) — explicit anti-affinity-by-AZ, balances load across school infrastructure.
- **Boot-from-Volume** flavors (`k8s.cp.vol`, `k8s.w.vol`) — persistent root disks survive node rebuilds.
- **Network**: `k8s-net` (`10.50.0.0/24`) + Calico VXLAN pod network `10.244.0.0/16`.
- **Security group `k8s-sg`** opens only what's needed:
  - `22/tcp` from admin CIDR (SSH)
  - `6443/tcp` (K8s API)
  - `10250-10259/tcp` kubelet + control-plane internal
  - `2379-2380/tcp` etcd internal
  - `4789/udp` Calico VXLAN

  - `30000-32767/tcp` NodePort range
- **Bootstrap = zero-touch**: cloud-init installs Ansible, then a `systemd` service+timer (retryable) clones `github.com/med-aziz-benamor/ansible-k8s` and provisions Kubernetes 1.29.15 + Calico across the fleet automatically.

2.2 Namespace + quotas — `k8s/base/`

- `piclouddoom` (prod) — quota: 50 pods, 14/28 CPU req/limit, 20/40 Gi mem, 10 PVCs.
- `piclouddoom-dev` (dev) — quota: 30 pods, 4/8 CPU, 6/12 Gi, 5 PVCs.

The prod quota was tuned twice during real cut-over (commits `83c29438`, original limits caused 4/7 services to fail to create a new `ReplicaSet` during the first rolling update because the temporary 2×replica spike exceeded memory headroom). This is documented inline in `k8s/base/resource-quota.yaml`.

2.3 Stateful infra — `k8s/infra/`

Every infra service is a single-replica `Deployment` with a `local-path` PVC (no replication today — see §6 limitations).

Component	Image	Storage	Notes
PostgreSQL	`postgres:16-alpine`	10Gi PVC	Init script via ConfigMap creates multiple DBs (`userdb`, `keycloakdb`, …). `pg_isready` probes.
Redis	`redis:7-alpine`	none (ephemeral)	Password-protected, 256MB maxmem, `allkeys-lru` eviction.
Kafka	`confluentinc/cp-kafka:7.5.0` (KRaft)	5Gi PVC	Single-broker, internal `:29092` + external `:9092`. **No ZooKeeper**. `kafka-topics-init` Job creates 14 topics.
MinIO	`minio/minio:2025-04-22`	5Gi PVC	S3-compatible object store. API `:9000` + console `:9001`.

Component	Image	Storage	Notes
Keycloak	`quay.io/keycloak/keycloak:24.0.1`	`keycloak-pvc`	Backed by Postgres `keycloakdb`. Realm `myapp-realm` imported from a ConfigMap-mounted JSON. Custom `interv` theme mounted via ConfigMap.

2.4 Application workloads — `k8s/apps/`

The 7 Spring Boot services follow a consistent pattern (see `k8s/apps/user-service/user-service.yaml` as the reference):

- **`Deployment` + `Service` (ClusterIP)** per app.
- **`strategy: Recreate`** for services that hold uploads/PVCs (`user-service`); rolling for the others.
- **Env injection split between ConfigMap (`app-config`) and Secret (`app-secrets`)** — never inline credentials.
- **Probes triple-stack**: `startupProbe` (lenient, 4-min budget), `readinessProbe`, `livenessProbe` — all hitting `/actuator/health`. Spring's slow JVM warmup means a misconfigured startup probe is the #1 cause of false rollback alarms, hence the long failureThreshold.
- **Resource requests/limits** sized per-service (e.g. user-service: 250m/512Mi → 1500m/1Gi).
- **Pinned images via `sha-<short>` tag** (e.g. `docker.io/azizbna/pi-clouddoom-user-service:sha-37`). The deploy pipeline rewrites this field on every deploy — see §4.
- **PVC where needed** (`user-uploads-pvc` for CV uploads).

2.5 Public ingress — `k8s/ingress/`

Traffic terminates outside the cluster on Cloudflare's edge, then enters via a **Cloudflare Tunnel** running in-cluster:

1. **Cloudflared deployment** (`k8s/ingress/cloudflared/cloudflared.yaml`) runs in the `traefik` namespace, opens an outbound tunnel to Cloudflare → no inbound public port is exposed on OpenStack.
2. Cloudflare forwards `api.interviewprep-tn.me` + `auth.interviewprep-tn.me` to Traefik.
3. **Traefik** is installed via Helm (`k8s/ingress/traefik-values.yaml`) and pinned to `k8s-w1` (the labeled `ingress-node`) using `nodeSelector`. It

binds `80/443` on the host with `hostPort` (no LoadBalancer service needed — OpenStack doesn't ship a cloud controller manager).

4. `api-ingress.yaml` routes paths (`/api/users`, `/api/community`, `/api/v1/training`, …) to the right ClusterIP service.
5. `auth-ingress.yaml` routes `auth.interviewprep-tn.me` → Keycloak.
6. **CORS Middleware** (`cors-middleware.yaml`) enforces `Access-Control-Allow-Origin` for the Vercel frontend + localhost dev, wired into ingresses via `traefik.ingress.kubernetes.io/router.middlewares: piclouddoom-cors-headers@kubernetescr`

This three-layer ingress (Cloudflare → Traefik → Service) gives us **free TLS** (Cloudflare edge), DDoS protection, and removes the need for a public IP on OpenStack — a major simplification.

2.6 Configuration management

Layer	What lives there
`app-config` ConfigMap	Public, non-sensitive endpoints: Keycloak issuer URI, Redis host, Kafka bootstrap, AI service URLs (Azure FQDNs), Mail server, model names.
`app-secrets` Secret	Postgres creds, Redis password, Groq API key, Google AI API key, Keycloak admin creds. **Hand-applied once** on `k8s-cp1` (not committed).
`keycloak-realm` ConfigMap	The Keycloak realm export JSON (mounted into the Keycloak pod).
`cloudflared-token` Secret	Cloudflare Tunnel token.
`minio-secrets` Secret	MinIO root creds.

2.7 Internal RBAC for CI/CD — `k8s/rbac/`

The pipeline never gets cluster-admin. It uses a **namespace-scoped** ServiceAccount:

- `cicd-deployer` SA in `piclouddoom` (prod) and `piclouddoom-dev` (dev).
- Role can `get/list/watch/patch/update` Deployments + ReplicaSets, read Pods/Services/Events/ConfigMaps.
- **Cannot** touch Secrets, RBAC, other namespaces, or anything cluster-scoped.
- Long-lived token kept in a `kubernetes.io/service-account-token` Secret (K8s 1.24+ no longer auto-mints these), extracted to a kubeconfig by `scripts/extract-kubeconfig.sh` and stored in GitHub secret `KUBECONFIG_PROD` / `KUBECONFIG_DEV`.

This is defence-in-depth: if the GitHub secret leaks, blast radius is limited to "redeploy app images in piclouddoom" — no exfiltration, no escalation.

2.8 Monitoring + observability — `k8s/monitoring/` + `k8s/attack-detection/`

- **kube-prometheus-stack** (Helm chart) installed via `monitoring-values.yaml`:
  - Prometheus (7-day retention), Alertmanager, Grafana (admin creds in values for now), kube-state-metrics, node-exporter.
  - `serviceMonitorSelector: {}` + cross-namespace selectors so any team can add a ServiceMonitor.
- **AI-based attack detector** (`k8s/attack-detection/`) — original component:
  - Custom Deployment `attack-detector` (image `docker.io/azizbna/pi-clouddoom-attack-detect` emits Prometheus metrics on `/metrics:8000`.
  - `ServiceMonitor` scrapes every 15s.
  - `PrometheusRule` fires `AIThreatDetected` alert when `attack_label == 1` over 1 min, plus `AttackDetectorDown` (critical) when metrics disappear for 2 min.
  - Two pre-built Grafana dashboards (`ai-attack-detector-k8s-dashboard.json`, `attack-detector-dashboard.json`).

---

3. Plane 2 — Azure Container Apps (AI services)

3.1 Why ACA, not OpenStack

The AI plane has very different characteristics from the Spring services: - **Heavy images** — `ollama` baked with `llama3.2:3b` is ~2 GB. - **Bursty traffic** — used only when a user runs a training-path generation or oral interview. - **High memory at peak**, idle most of the time.

ACA gives **scale-to-zero**, per-revision deployments, and built-in HTTPS ingress without us managing a single VM. We pay only for active CPU time. This would be wasteful on our small OpenStack cluster (4 workers).

3.2 The 3 AI services

Source folder	ACA app name	What it does
`ai-training-path/`	`ai-training-path`	FastAPI; predicts personalized training paths from user CV/profile vectors.
`kokoro/`	`kokoro`	TTS engine for the live mock-interview avatar.

Source folder	ACA app name	What it does
`ollama-deploy/`	`ollama`	LLM runtime serving `llama3.2:3b` — used by interview-service and community-service.

`whisper` (STT) also lives in this plane, but uses the upstream third-party image `fedirz/faster-whisper-server:latest-cpu` — we don't build it, so it's not in our deploy matrix. The Spring services know its URL via the `WHISPER_BASE_URL` ConfigMap key.

3.3 Resource group + identity

- Resource group: **`rg-piclouddoom`** (referenced via GitHub secret `AZURE_RESOURCE_GROUP`).
- Region: **France Central** (low latency from Tunis).
- Identity: a service principal (`AZURE_CREDENTIALS` GitHub secret, `--sdk-auth` JSON) scoped to **`Contributor` on the RG only** — never on the full subscription.
- **Known constraint** (`docs/cicd/azure-sp-blocked.md`): creating the SP requires Entra ID admin grant from esprit.tn, which is currently pending. Workaround: `scripts/deploy-aca.sh` is a manual fallback that uses `az login` from a developer laptop and replicates the workflow steps.

3.4 Deployment mechanism

For each AI service, the pipeline runs:

```
az containerapp update \
  --name <app> \
  --resource-group rg-piclouddoom \
  --image docker.io/azizbna/pi-clouddoom-<svc>:sha-<short> \
  --revision-suffix sha-<short>
```

This creates a **new revision**. Single-revision mode (default) routes 100% of traffic to the new revision immediately, ACA orchestrates pod-level cutover with health checks. We then **poll the revision's `runningState`** every 10s for up to 5 min: - `Running` / `RunningAtMaxScale` → success. - `Failed` / `Degraded` → fail the job. - Timeout → fail.

On failure, an **auto-rollback** step lists prior `Running` revisions, picks the newest non-broken one, and swings traffic via `az containerapp ingress traffic set --revision-weight <prev>=100`.

3.5 What we *don't* manage from CI

`az containerapp update --image` only updates the image, not env vars or secrets. The container app's env config (e.g. `GROQ_API_KEY` if any AI service consumes it directly) is managed once via `az containerapp secret set` / portal. Migrating to Bicep is on the post-launch backlog.

---

4. Plane 3 — Vercel (frontend)

4.1 The deployment surface

- **Project ID**: `prj_2vhl5NT0TAWAQZkKNWPm3UKs25HL` (`team_tQYcRewNfvSBGAbsE6wJwcop`).
- **Production URL**: `https://interviewprep-tn.me`.
- **Build config** — `vercel.json`:

```
{
  "buildCommand": "cd frontend && npm ci && npm run build -- --configuration production
  "outputDirectory": "frontend/dist/interviewpreptn/browser",
  "installCommand": "cd frontend && npm ci",
  "framework": "angular",
  "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }]
}
```

  The SPA `rewrites` rule routes all non-asset paths back to `index.html` so Angular's client-side router handles them.

- `.vercelignore` excludes every backend folder (`user-service/`, `community-service/`, `infra/`, …) and Postman collections, so Vercel only sees the Angular sources. Cuts build context dramatically.

4.2 Two-channel deploys

Trigger	Workflow	Effect
PR touching `frontend/**`	`vercel-preview.yml`	Builds + deploys a preview URL; posts a sticky PR comment with the URL.
Push to `main` touching `frontend/**`	`vercel-prod.yml`	Builds + deploys to `interviewprep-tn.me`; smoke-checks `200 OK` with retry.

We deliberately **don't** use Vercel's native GitHub auto-integration — the GitHub Actions workflow is the single source of truth. That gives us: - One

audit trail (Actions log). - Pre-deploy gates (CI must pass first via branch protection). - Identical preview + prod flow controlled in YAML, not Vercel's UI.

4.3 Vercel CLI flow

Both workflows use the raw CLI, no third-party action:

```
vercel pull   --yes  --environment=<preview|production>  --token=$VERCEL_TOKEN
vercel build  [--prod]                                   --token=$VERCEL_TOKEN
vercel deploy --prebuilt [--prod]                        --token=$VERCEL_TOKEN
```

`--prebuilt` uploads the local output directly — no rebuild on Vercel's side, faster and deterministic.

---

5. Container image strategy

5.1 Registry layout — DockerHub `azizbna/pi-clouddoom-*`

All 10 backend/AI images live in public DockerHub repos:

#	Image	Source
1	`pi-clouddoom-user-service`	`user-service/`
2	`pi-clouddoom-interview-service`	`interview-service/`
3	`pi-clouddoom-training-service`	`training-service/`
4	`pi-clouddoom-mentorship-service`	`mentorship-service/`
5	`pi-clouddoom-quiz-service`	`quiz-service/`
6	`pi-clouddoom-community-service`	`community-service/`
7	`pi-clouddoom-resource-service`	`resource-service/`
8	`pi-clouddoom-ai-training-path`	`ai-training-path/`
9	`pi-clouddoom-kokoro`	`kokoro/`
10	`pi-clouddoom-ollama`	`ollama-deploy/`
(+)	`pi-clouddoom-attack-detection`	optional component, built separately

Public = no `imagePullSecret` to manage in the cluster.

5.2 Tagging convention

Tag	Meaning	Used by
`sha-<7-char>`	**Immutable**, unique per commit	Every deploy workflow pins this tag — the only safe deploy target

Tag	Meaning	Used by
`<branch>`	Moving pointer per branch (`main`, `deployment-hybrid-…`)	Humans, debugging
`latest`	Updated only on `main`	Convenience for `docker pull`
`v*.*.*`	Set on git tag push	Release tagging
`fixed`	Legacy mutable tag (left in place until full cut-over)	Pre-pipeline pods still pin this

5.3 Supply chain integrity

Every image build: 1. **Trivy scan** (`severity: CRITICAL,HIGH`, `ignore-unfixed: true`) → SARIF uploaded to GitHub Code Scanning. Currently report-only (`exit-code: "0"`) until the day-1 backlog of base-image CVEs is triaged; flip to `"1"` once clean. 2. **Cosign keyless signing** via GitHub OIDC — no private key to manage. Anyone can verify: `bash cosign verify docker.io/azizbna/pi-clouddoom-user-service:sha-<short> \ --certificate-identity-regexp "https://github.com/Mohamed-Fedi-Hamrouni/Pi-CloudDOOM/.*" \ --certificate-oidc-issuer https://token.actions.githubusercontent.com` 3. **SBOM** (`sbom: true`) + **Build provenance** (`provenance: mode=max`) attached as OCI attestations by `docker/build-push-action@v6`.

5.4 Build performance

- `docker/setup-buildx-action@v3` enables BuildKit.
- Per-service GHA cache: `cache-from: type=gha,scope=<service>` + `cache-to: type=gha,scope=<service>,mode=max` — unchanged service re-builds in ~30 s.
- Matrix runs 10 services in parallel with `fail-fast: false` — one broken service doesn't kill the others.

---

6. Deployment workflow per service type

6.1 Spring services (OpenStack)

```
1.  Build & Push images workflow finishes → publishes sha-<short> tag
2.  Deploy - OpenStack K8s auto-fires (workflow_run on success)
3.  gate job:
    - Resolves SHA (workflow_run head_sha or manual input)
    - Resolves environment (prod default, dev only via workflow_dispatch)
    - Picks namespace + kubeconfig secret
4.  deploy matrix (7 jobs, fail-fast: false), per service:
```

```
    - kubectl set image deployment/<svc> <svc>=<image>:sha-<short>
    - kubectl rollout status --timeout=10m
    - On failure: dump describe + logs + pods, then rollout undo
5.  smoke job (after all 7 succeed):
    - scripts/smoke.sh --kubeconfig=... --namespace=...
    - In-cluster mode: kubectl wait --for=condition=Available on each Deployment
6.  On smoke failure → ROLL BACK ALL 7 SERVICES in the namespace
```

6.2 AI services (Azure)

```
1.  Build & Push images workflow finishes
2.  Deploy - Azure Container Apps fires (currently manual workflow_dispatch only -

    waiting on Entra ID grant; scripts/deploy-aca.sh is the manual fallback)
3.  gate job → resolves SHA
4.  deploy matrix (3 jobs):
    - az containerapp update --image ... --revision-suffix sha-<short>
    - Poll runningState up to 5 min
    - On Failed/Degraded → traffic-shift back to previous Running revision
```

6.3 Frontend (Vercel)

```
PR opened touching frontend/**:
  vercel pull preview → vercel build → vercel deploy --prebuilt
  → sticky PR comment with preview URL

Push to main touching frontend/**:
  vercel pull production → vercel build --prod → vercel deploy --prebuilt --prod
  → curl interviewprep-tn.me, 5× retry on non-200
```

---

7. Rollback strategy

Rollback is multi-layered, every layer is automatic where possible:

Layer	Trigger	Action
Single-service rollout fails	`kubectl rollout status` non-zero	`kubectl rollout undo deployment/<svc>` + re-wait 3 min
Post-deploy smoke fails	`smoke.sh` exits non-zero	`kubectl rollout undo` on **all 7 services** in the namespace

Layer	Trigger	Action
ACA revision unhealthy	Polled state = Failed/Degraded, or 5-min timeout	`az containerapp ingress traffic set --revision-weight <prev>=100`
Vercel prod smoke fails	5× curl returns non-200	Job fails; previous production deployment continues serving (Vercel never replaces a deployment until the new one is live)
Manual rollback to any SHA	`Deploy - OpenStack K8s` → `workflow_dispatch` with `sha: <old-7-char>`	Re-pins the image tag, re-rolls
Emergency from `k8s-cp1`	shell access	`kubectl -n piclouddoom rollout undo deployment/<svc>`

Because every image is signed and immutable per commit, "rolling back" is just "pinning a different `sha-<short>` tag" — there's no state to recover.

---

8. Domain + TLS strategy

- **Cloudflare** manages DNS for `interviewprep-tn.me` + subdomains.
- **TLS termination** happens at Cloudflare's edge (free, auto-renewed).
- Cloudflare → Cloudflare Tunnel (cloudflared in cluster) → Traefik → Service. No public-facing certificate to manage inside the cluster.
- Vercel issues its own auto-managed cert for the apex on `interviewprep-tn.me`.

---

9. Day-1 known limitations & post-launch backlog

(Documented in `docs/cicd/README.md`; reproduced here for completeness.)

1. **No HA**: single replica per app, single Postgres, single Kafka, single K8s control-plane node.
2. **No GitOps**: deploys are imperative (`kubectl set image`). Argo CD is on the backlog.

3. **No DB migration coordination**: Flyway runs at Spring boot startup. Incompatible migrations crash-loop the new pod → auto-rollback handles it but is loud. Pre-migration tooling needed.
4. **IaC scan is report-only**: flip `exit-code: "1"` in `iac-scan.yml` after triaging backlog (mostly missing PDBs and single-replica warnings).
5. **Smoke test covers Spring services only**: extend to ACA URLs + Vercel post-launch.
6. **Azure SP expires** annually — migrate to federated OIDC credentials when the SP can be created (currently blocked).
7. **Dev namespace is empty** — needs `k8s/apps/*` Kustomize overlays applied.
8. **No PodDisruptionBudgets** today — `kubectl drain` would temporarily drop replicas.
9. **Single replica per service** rolling updates use `Recreate` for stateful svc (`user-service`) and roll naturally for the rest.

---

10. One-time bootstrap checklist (for a fresh environment)

These steps were performed once per cluster. If you re-deploy from scratch:

```
# OpenStack side, on k8s-cp1
kubectl create namespace piclouddoom
kubectl apply -f k8s/base/resource-quota.yaml
kubectl apply -f k8s/base/app-config.yaml
# hand-create app-secrets, minio-secrets, cloudflared-token, keycloak-realm CM, ...
kubectl apply -f k8s/infra/postgres/
kubectl apply -f k8s/infra/redis/
kubectl apply -f k8s/infra/kafka/
kubectl apply -f k8s/infra/kafka/kafka-topics-job.yaml
kubectl apply -f k8s/infra/minio/
kubectl apply -f k8s/infra/keycloak/
kubectl apply -f k8s/apps/              # all 7 services
kubectl label node k8s-w1 ingress-node=true
helm install traefik traefik/traefik -n traefik --create-namespace \
  --skip-crds -f k8s/ingress/traefik-values.yaml
kubectl apply -f k8s/ingress/cors-middleware.yaml
kubectl apply -f k8s/ingress/api-ingress.yaml
kubectl apply -f k8s/ingress/auth-ingress.yaml
kubectl apply -f k8s/ingress/cloudflared/

# Monitoring
helm install monitoring prometheus-community/kube-prometheus-stack \
  -n monitoring --create-namespace -f k8s/monitoring/monitoring-values.yaml
kubectl apply -f k8s/attack-detection/namespace.yaml
kubectl apply -f k8s/attack-detection/
```

```
# CI/CD RBAC
kubectl apply -f k8s/rbac/cicd-deployer.yaml
bash scripts/extract-kubeconfig.sh > /tmp/kubeconfig-prod.yaml
# → paste as GitHub secret KUBECONFIG_PROD, then shred
```

For Azure:

```
az group create -n rg-piclouddoom -l francecentral
# Create ACA environment + 3 container apps (one-time)
# Then CI takes over via az containerapp update
```

For Vercel:

```
vercel link        # already done - .vercel/project.json is committed
# Add VERCEL_TOKEN to GitHub secrets
```

---

11. Summary table

Concern	Solution
Cluster provisioning	OpenStack Heat (`infra/openstack/heat-cluster.yaml`) + Ansible bootstrap
Workload orchestration	Kubernetes 1.29.15 + Calico CNI
Container registry	DockerHub public (`docker.io/azizbna/pi-clouddoom-*`)
Image security	Trivy scan + Cosign keyless signing + SBOM + provenance
Backend deploy	`kubectl set image` → rollout status → auto-undo, gated by `Build & Push`
AI deploy	`az containerapp update` → revision health poll → traffic shift back on fail
Frontend deploy	Vercel CLI (preview on PR, prod on `main`)
Public TLS / DNS	Cloudflare + Cloudflare Tunnel (cloudflared)
In-cluster ingress	Traefik (Helm) pinned to ingress-node, hostPort 80/443
Auth	Keycloak 24 with realm-export in ConfigMap
Stateful infra	Postgres, Redis, Kafka (KRaft), MinIO — each 1×replica with PVC

Concern	Solution
Observability	kube-prometheus-stack (Prometheus, Grafana, Alertmanager) + custom AI attack detector
Multi-env	`piclouddoom` (prod) + `piclouddoom-dev` (dev) namespaces with separate quotas + RBAC
Rollback	Auto-undo on rollout failure, full-namespace rollback on smoke failure, manual `workflow_dispatch` `sha:` for any historical SHA